



2026-2학기 데이터사이언스

5주차. 데이터 구하기 도시 데이터 수집과 git · github

2026. 10. 5.

그 증거의 원료는 어디에 있는가?

- 지난주 문제정의서에 "이런 데이터가 필요하다" 고 적었다
- 그게 실제로 있는가
- 있다면 어떻게 손에 넣는가

국내 데이터는 크게 아홉 군데에 있다

어디	무엇이 있나	받는 방식
KOSIS 국가통계포털	인구·가구·산업 통계표	화면 조회 · CSV · MCP
공공데이터포털	부처·지자체가 여는 거의 전부	파일 · API
SGIS 통계지리정보	격자·행정동 단위 공간통계	화면 · API
실거래가 공개시스템	부동산 매매·전월세 거래	CSV · API
브이월드 <code>vworld.kr</code>	2D·3D 국가공간정보 — 용도지역지구·건축물·지형·경사도	지도 조회는 가입 없이 · 다운로드와 오픈API는 가입 후 인증키
국토정보플랫폼 <code>map.ngii.go.kr</code>	수치지도·항공사진·정사영상·국토통계지도. 대동여지도·북한지도 같은 역사자료도 있다	열람은 무료 · 내려받기는 가입 후 신청 (일부는 공문)
KTDB 국가교통DB <code>ktdb.go.kr</code>	여객·화물 통행실태조사, 교통통계DB (수송실적·교통시설 규모·접근성지표)	자료신청 → 심사 뒤 제공 . 즉시 받지 못한다
COMPAS	과제별로 정제된 도시 데이터	과제 단위 일괄
지자체 데이터포털	그 도시에만 있는 것	제각각

받는 방식이 곧 일정이다. 위 넷은 오늘 당장 받을 수 있고, 국토정보플랫폼·KTDB 는 신청부터 해 두어야 한다.
오늘은 **실거래가** 하나를 세 방식으로 받아 본다 — 방식을 익히면 나머지도 같다.

같은 데이터를 얻는 세 가지 방식

① 수동 다운로드

화면에서 조건을 골라 받는다

한 번은 제일 빠르다
열두 달이면 열두 번

② API 호출

인증키로 프로그램이 받는다

키 발급과 규격 이해가 필요
한 번 만들면 반복이 공짜

③ AI 위임

조건만 주고 맡긴다

규격을 몰라도 된다
받은 것이 맞는지 봐야 한다

손이 많이 간다

손이 덜 간다 · 확인할 것이 는다

어느 하나가 옳은 게 아니다 — **몇 번 쓸 것인지**로 고른다

① 손으로 받는다 — 상당구 2026년 8월

- 1 실거래가 공개시스템 `rt.molit.go.kr` → 자료제공 → 조건별 자료제공
- 2 탭은 아파트, 거래는 매매
- 3 계약일자 **2026-08-01 ~ 2026-08-31** · 시도 **충청북도** · 시군구 **청주시 상당구**
- 4 EXCEL 다운 또는 CSV 다운

주소창에 이것을 넣으면 오른쪽 화면이 바로 열린다

`https://rt.molit.go.kr/pt/xls/xls.do?mobileAt=`

다만 이 주소는 **조건이 비어 있는 화면**이다. 고른 기간과 지역은 주소에 남지 않아서 — 이 주소만 적어 두어서는 같은 파일을 다시 받지 못한다.

국토교통부 실거래가 공개시스템

아파트 | 연립/다세대 | 단독/다가구 | 오피스텔 | 토지 | 분양/입주권 | 상업/업무용 | 공장/창고 등

조건별 자료제공

조건별 자료제공 이용시 유의사항

본 서비스에 제공하는 정보는 법적인 효력이 없으므로 참고용으로만 활용하시기 바라며, 외부 공개시에는 반드시 신고일 기준으로 집계되는 공식통계를 이용하여 주시기 바랍니다.

신고정보가 실시간 변경, 해제되어 제공시점에 따라 공개건수 및 내용이 상이할 수 있는 점 참고하시길 바랍니다.

본 자료는 계약일 기준입니다. (7월 계약, 8월 신고건 → 7월 거래건으로 제공)

- 주택매매 거래는 부동산 거래신고 등에 관한 법률 제3조에 따라 계약일로부터 30일 이내 신고토록 하고 있습니다.
- 시도별 자료제공 계약일자 범위는 최대 1년입니다. 이용에 참고하시길 바랍니다.
- 계약일자 범위 내 신고정보가 없는 경우 단지 및 지번 목록이 제공되지 않습니다. 조회량이 많을 경우 다운로드 시간이 다소 소요될 수 있습니다.

아파트 | 연립/다세대 | 단독/다가구 | 오피스텔 | 분양/입주권 | 상업/업무용 | 토지 | 공장/창고 등

매매 | 전월세

계약일자: 2026-08-01 ~ 2026-08-31

지번구분: 지번주소 | 도로명주소 | 시도: 충청북도 | 시군구: 청주시 상당구

읍면동: 전체 | 단지명: 전체

면적: 전체 | 금액: 최소금액(만원) ~ 최대금액(만원)

EXCEL 다운 | CSV 다운

국토교통부 실거래가 공개시스템 「조건별 자료제공」 · 2026. 9. 28. 갈무리. 자료는 **계약일 기준**이라고 화면이 먼저 알린다

API 는 어렵지 않다 — 주소에 조건을 붙이는 것이다

국토교통부_아파트 매매 실거래가 자료 · `getRTMSDataSvcAptTrade`

붙이는 조건	뜻	예
<code>serviceKey</code>	나를 알아보는 열쇠	포털에서 발급받는다
<code>LAWD_CD</code>	지역 — 법정동코드 앞 5자리	43111 (청주시 상당구)
<code>DEAL_YMD</code>	계약 연월	202608

앞 장과 같은 자료를 API 로 — 주소 하나에 조건이 다 들어 있다

```
https://apis.data.go.kr/1613000/RTMSDataSvcAptTrade/getRTMSDataSvcAptTrade
?serviceKey=●●●&LAWD_CD=43111&DEAL_YMD=202608
```

→ 청주시 상당구 2026년 8월 아파트 매매 **182건**

앞 장의 주소에는 없던 것이 여기 다 있다. 이 한 줄을 적어 두면 같은 파일이 언제든 다시 나온다. 조건 셋을 바꾸면 다른 지역·다른 달이고, 열 두 달은 열두 번 부르면 된다 — 그것이 프로그램이 할 일이다.

무료이고 활용신청은 자동 승인, 개발계정은 하루 10,000건까지다. 법정동코드는 code.go.kr 에서 찾는다 — **코드를 찾는 일 자체가 오늘 배울 기술이다.**

③ 맡긴다 – 같은 182건을 클로드가 받는다

주는 것은 둘뿐이다 – 문서 주소와 인증키

이렇게 맡긴다

"공공데이터포털 국토교통부 아파트 매매 실거래가 API 를 써 줘. 문서는 `data.go.kr/data/15126469/openapi.do` 고 인증키는 따로 줄게.

LAWD_CD 43111(청주시 상당구) · DEAL_YMD 202608 로 받아서 **CSV 한 장**으로 만들어 줘. **몇 건이 왔는지 먼저 알려 주고** 시작해."

규격을 내가 읽지 않아도 된다. **다만 건수는 내가 안다** – ①과 ②에서 182건을 이미 봤다.

한 번 쓰고 말 것인가, 매달 쓸 것인가

한 번 쓴다면	매달 쓴다면
대화로 시킨다	스킬로 만든다 – 3주차에 배운 것
그때그때 프롬프트를 다시 쓴다	「상당구 지난달 실거래가」 한 마디면 된다
무엇을 시켰는지 남지 않는다	<code>SKILL.md</code> 에 절차가 남는다

스킬로 만들면 그것이 ②다. 지역코드와 연월만 바꿔 같은 절차를 다시 돌리는 것 – 그게 API 를 부르는 일이다. ③은 ②를 없애는 것이 아니라 ②를 쓰기 쉽게 만든다.

인증키조차 필요 없는 경우도 있다

3주차에 붙인 KOSIS MCP

앞의 실거래가는 **인증키를 받아** API 를 불렀다. KOSIS 통계표에는 그 단계가 없다 — 주소 하나로 붙는 MCP 서버를 시범 운영한다. 1,360여 종 국가승인통계, 26만여 통계표가 대상이다.

통계표 번호를 미리 알 필요도 없다. `kosis_search` 가 표를 찾고 `kosis_region_code` 가 청주시 = `43110` 을 돌려준다. 그 둘을 `kosis_get_data` 에 넣으면 숫자가 온다.

다만 **시범서비스이며 국가데이터처의 공식 입장이 아니다**. 중요한 판단 전에는 KOSIS 원자료를 확인한다 (KOSIS 공지, 2026-07-08). 뒤에서 이 서버로 청주시 인구를 받아 본다.

맡겼으면 확인한다

①②③ 이 같은 182건을 내놓아야 한다. 어긋나면 거기서 멈춘다

무엇을 보나	왜	어긋나면
건수	①에서 화면으로 센 182건이 기준이다	기간을 어떻게 읽었는지, 취소 거래를 넣었는지 묻는다
어디서 왔나	API 이름과 제공기관을 되묻는다	대지 못하면 그 숫자는 쓰지 않는다
자릿수가 그럴듯한가	상당구 한 달에 182건이면 하루 여섯 건꼴이다	자릿수가 어긋나면 합산을 잘못된 것이다

자릿수 확인은 실제로 쓰인다. 뒤에 나오는 청주시 인구에서 처음 합산이 167만으로 나왔다. 받아 온 표에 「계」 행과 연령별 행이 함께 있었고 그것을 모르고 다 더한 것이다. 청주시가 85만 규모라는 것만 알아도 걸린다.

①②를 해 본 사람만 ③을 의심할 수 있다. 세 번 받아 보는 이유가 이것이다.

직접 받기 — 청주시 인구 (2000~2024)

질문은 하나다 — 청주시 인구는 25년 동안 어떻게 변했나?

- 1 `kosis.kr` → 국내통계 → 주제별 통계
- 2 인구 → 주민등록인구현황 → 행정구역(시군구)별, 성별 인구수
- 3 표가 열리면 조회설정에서 셋을 고른다 — 행정구역 충청북도 > 청주시 · 항목 총인구수 · 시점 2000 ~ 2024 (년)
- 4 다운로드 → CSV 또는 EXCEL

AI에 맡기기 – 청주시 인구 (2000~2024)

이렇게 묻는다

"청주시 총인구를 2000년부터 2024년까지 연도별로 받아 줘. 어느 통계표에서 왔는지도 같이 알려 줘."

클로드가 한 일 – 다섯 번 불렀다

- 1 kosis_local_search 로 표를 찾음 → DT_1B04006, 그런데 2008년부터다
- 2 kosis_search 로 다시 찾음 → DT_1B040A3, 1992년부터 ✓
- 3 kosis_region_code 로 청주시 = 43110
- 4 kosis_get_data — 실패 항목ID 를 T2 로 넣었다. 유효값은 T20·T21·T22 였다
- 5 kosis_get_data — 25줄이 왔다

표를 잘못 짚는 것은 어느 쪽에서나 같다 — 맡기면 2번에서 걸러진다.

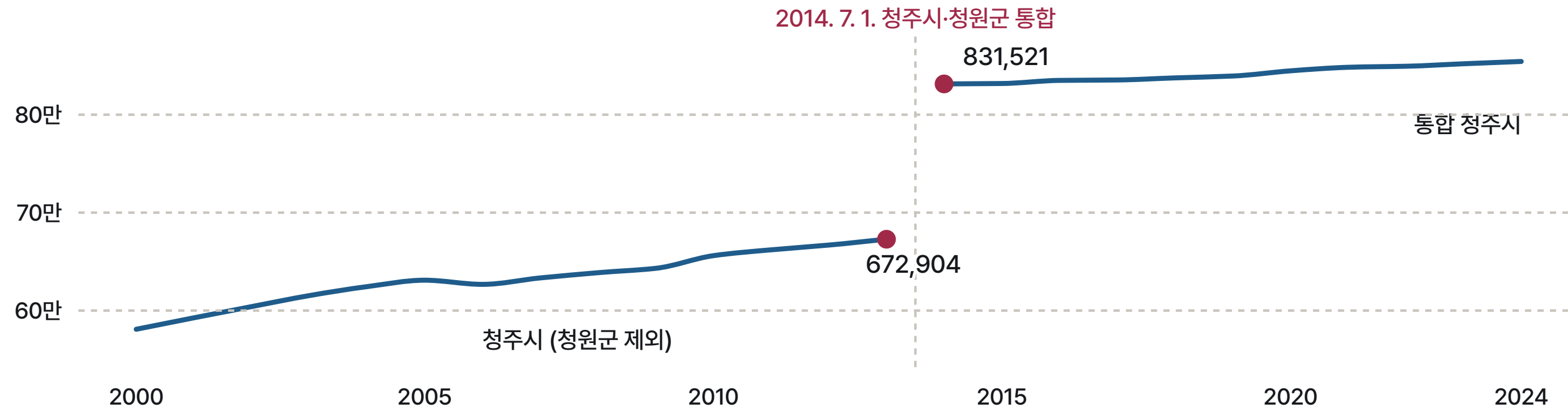
돌아온 값 (일부)

연도	총인구(명)
2000	580,861
2005	630,939
2010	655,971
2013	672,904
2014	831,521
2020	844,993
2024	854,537

2013 → 2014 에 158,617명이 늘었다. 1년 만에 23.6%다. 이런 숫자가 나오면 멈춘다.

출처: KOSIS 「행정구역(시군구)별, 성별 인구수」(행정안전부, DT_1B040A3), KOSIS MCP 로 2026-09-28 조회

둘 다 맞는 숫자다 – 도시가 바뀐 것이다



2013년 청주시 672,904 + 청원군 155,002 = 827,906. 이듬해 통합 청주시가 831,521 이니 한 해치 증가분만큼 차이가 난다. **인구가 났던 것이 아니라 시의 경계가 넓어진 것이다.** 청원군은 2014년부터 이 표에서 사라진다.

청주시 + 청원군 통합을 모른다면 2013~2014년의 차이를 그대로 받아들인다. AI 를 사용하면 오히려 오류를 범할 확률이 줄어들 수 있다.

받고 나서 겪는 것들

겪는 것	무슨 일인가
글자가 깨진다	공공기관 CSV 는 EUC-KR 인 경우가 많다. 엑셀은 열리는데 파이썬에서 깨지면 이것이다
숫자가 글자다	거래금액에 쉼표가 들어 있다 — "1,250" 은 숫자가 아니다
지역이 안 맞는다	행정동과 법정동은 다르다. 통계는 행정동, 부동산은 법정동인 경우가 많다
기간이 비어 있다	거래가 없는 달은 행이 아예 없다. 0 이 아니라 없음 이다
단위가 다르다	거래금액은 만원, 면적은 m². 평으로 바꾸면 어디선가 어긋난다

전부 클로드가 처리해 준다. **다만 처리했다는 사실을 알고 있어야** 결과가 이상할 때 어디를 볼지 안다.

출처를 적지 않은 데이터는 증거가 아니다

데이터만 받아 두면 증거가 되지 않는다 —
메타데이터를 함께 정리한다

출처 기관과 주소 · 받은 날짜 · 대상 기간과 지역 · 단위 · 집계 기준 · 결측을 어떻게 뒀나 · 알려진 한계

→ 석 달 뒤의 나는 이 파일이 어디서 왔는지 기억하지 못한다.

남이 물었을 때 답하지 못하면 그 분석은 쓸 수 없다

실습

청주 상당구 실거래가를 세 방식으로 받기 — 약 1시간 30분

실습 - 오늘 만들 것

대상

청주시 상당구 아파트 매매 실거래가 · 최근 12개월

원도심이 있는 구다. 4주차에 정한 연구질문과 이어지고, 6주차부터 내내 이 파일을 쓴다.

왜 세 번이나 받나

- 같은 데이터가 나와야 **내가 제대로 받았는지** 알 수 있다
- 세 방식의 손이 가는 정도를 몸으로 비교하게 된다
- ③ 이 편하다는 것을 알되, ①②를 해 봐야 **③의 결과를 의심할 수 있다**

실습 ① – 손으로 받는다 20분

- 1 국토교통부 실거래가 공개시스템 `rt.molit.go.kr` 에 들어간다
- 2 아파트 → 매매 → 지역을 충청북도 청주시 상당구로, 기간을 최근 12개월로 지정한다
- 3 화면에서 결과를 훑어본다 — 몇 건인가
- 4 CSV 로 내려받아 열어 본다. 열 이름과 단위를 확인한다

메모해 둘 것: 전체 건수. ②③의 결과를 이것과 맞춰 본다.

조건을 하나씩 바꿔 보면 무엇이 어떻게 걸러지는지 보인다. API 의 파라미터가 곧 이 화면이다.

실습 ② – 인증키를 받는다 35분

- 1 `data.go.kr` 회원가입 (공공데이터포털)
- 2 "국토교통부_아파트 매매 실거래가 자료" 를 찾아 **활용신청**한다. 자동 승인이라 곧 쓸 수 있다
- 3 마이페이지에서 **인증키(serviceKey)**를 확인한다. 상세 화면에 요청 주소와 파라미터 설명이 함께 있다
- 4 `code.go.kr/stdcode/regCodeL.do` 에서 법정동코드를 찾는다 — 청주시 상당구 `4311100000` → 앞 5자리 **43111**
- 5 서원구·흥덕구·청원구의 코드도 직접 찾아 적어 둔다

인증키는 남에게 보이지 않는다. 화면 공유나 제출물에 그대로 넣지 않는다.

실습 ③ – 말긴다 20분

프롬프트 예시

"공공데이터포털의 국토교통부 아파트 매매 실거래가 API를 쓰려고 해. 인증키는 따로 알려줄게.

지역은 청주시 상당구(LAWD_CD 43111), 기간은 최근 12개월이야. 열두 달을 각각 받아서 CSV 한 장으로 합쳐 줘.

합치기 전에 어떻게 받아서 어떻게 합칠지 먼저 알려 줘. 다 받은 뒤에는 달마다 몇 건이 왔는지 표로 보여 줘."

마지막 두 문장이 핵심이다. **받기 전에 방법을, 받은 뒤에 건수를** 요구한다 — 그래야 ①에서 센 건수와 맞춰 볼 수 있다.

건수가 다르면 멈춘다. 기간 해석이 다른지, 취소 거래를 포함했는지 물어본다.

MCP 로도 한 번 받아 본다. 3주차에 붙인 **KOSIS MCP** 로 자기 연구질문에 필요한 통계표를 하나 받는다. 인증키가 없어 바로 된다. 받은 뒤 통계표ID 와 기관을 되묻는 것까지가 실습이다.

실습 ④－출처를 적는다 ^{15분}

데이터 옆에 메타데이터를 같이 둔다 — 앞 장의 일곱 줄이다

칸	예
출처	국토교통부 아파트 매매 실거래가 자료 (공공데이터포털 API)
주소	<code>apis.data.go.kr/1613000/RTMSDataSvcAptTrade/getRTMSDataSvcAptTrade</code>
받은 날	2026. 10. 5.
범위	청주시 상당구 (LAWD_CD 43111), 2025. 10. ~ 2026. 9. 계약분
단위	거래금액 만원 · 전용면적 m ²
집계 기준	계약일 기준이다 (신고일 기준이 아니다)
결측·한계	거래 없는 달은 행이 아예 없다. 신고 시차가 있고, 취소 거래 처리 방식은 확인 필요

제출: cheongju-apt.csv + 출처 문서 — 다음 주에 이 파일을 저장소에 올리고, 7주차에 이 파일로 첫 분석을 한다.

과제: 자기 문제정의서에 필요한 데이터도 하나 찾아 둔다. 없으면 없다고 적어 온다.

그리고 다음 장의 사전 과제를 반드시 해 온다.

자료 출처

- 국토교통부_아파트 매매 실거래가 자료 (오퍼레이션 `getRTMSDataSvcAptTrade`)
data.go.kr/data/15126469/openapi.do
- 국토교통부 실거래가 공개시스템 — rt.molit.go.kr
- 법정동코드 · 청주시 상당구 4311100000 — 행정표준코드관리시스템
code.go.kr/stdcode/regCodeL.do (2026. 8. 26. 조회)
- KOSIS 「행정구역(시군구)별, 성별 인구수」(행정안전부, `DT_1B040A3`)
kosis.kr/statHtml/statHtml.do?orgId=101&tblId=DT_1B040A3 (2026. 9. 28. 조회)
- 실거래가 공개시스템 「조건별 자료제공」 화면 갈무리 (5쪽) — 2026. 9. 28. 갈무리
rt.molit.go.kr/pt/xls/xls.do?mobileAt=
- 브이월드 (국토교통부 · 공간정보산업진흥원) — vworld.kr
국토정보플랫폼 (국토지리정보원) — map.ngii.go.kr (2026. 9. 28. 확인)
- 그 밖의 소스 — KOSIS kosis.kr · 공공데이터포털 data.go.kr · SGIS sgis.kostat.go.kr · KTDB ktdb.go.kr · COMPAS compas.lh.or.kr

분석_최종_진짜최종_v3.py

- 어제 잘 되던 분석이 오늘 안 된다. 무엇을 바꿨더라
- AI가 파일을 고쳤는데 되돌리고 싶다
- 제출한 결과를 남이 그대로 재현할 수 있어야 한다

AI와 함께 일할수록 더 필요해진다

클로드는 파일을 실제로 고친다.

고치기 전 상태가 어딘가 남아 있어야 마음 놓고 시킬 수 있다

→ 깃은 되돌릴 수 있게 해 주고, 깃허브는 그것을 남에게 보여 줄 수 있게 한다.
증거를 다루는 사람에게 이것은 편의가 아니라 **요건**이다

오늘 배울 세 가지

무엇	해결하는 것	이 수업에서의 쓸모
① 깃	내 컴퓨터 안의 기록과 되돌리기	AI 가 고친 것을 되돌린다
② 깃허브	남과 나누기와 백업	분석을 제출하고 재현시킨다
③ 클로드 + 깃	커밋·PR 을 맡기기	기록을 남기는 품을 줄인다

깃 명령을 외우는 수업이 아니다. 무엇을 왜 기록하는지를 배우고 명령은 클로드에게 맡긴다.

저장한다 ≠ 기록한다

그냥 저장하면



커밋을 쌓으면

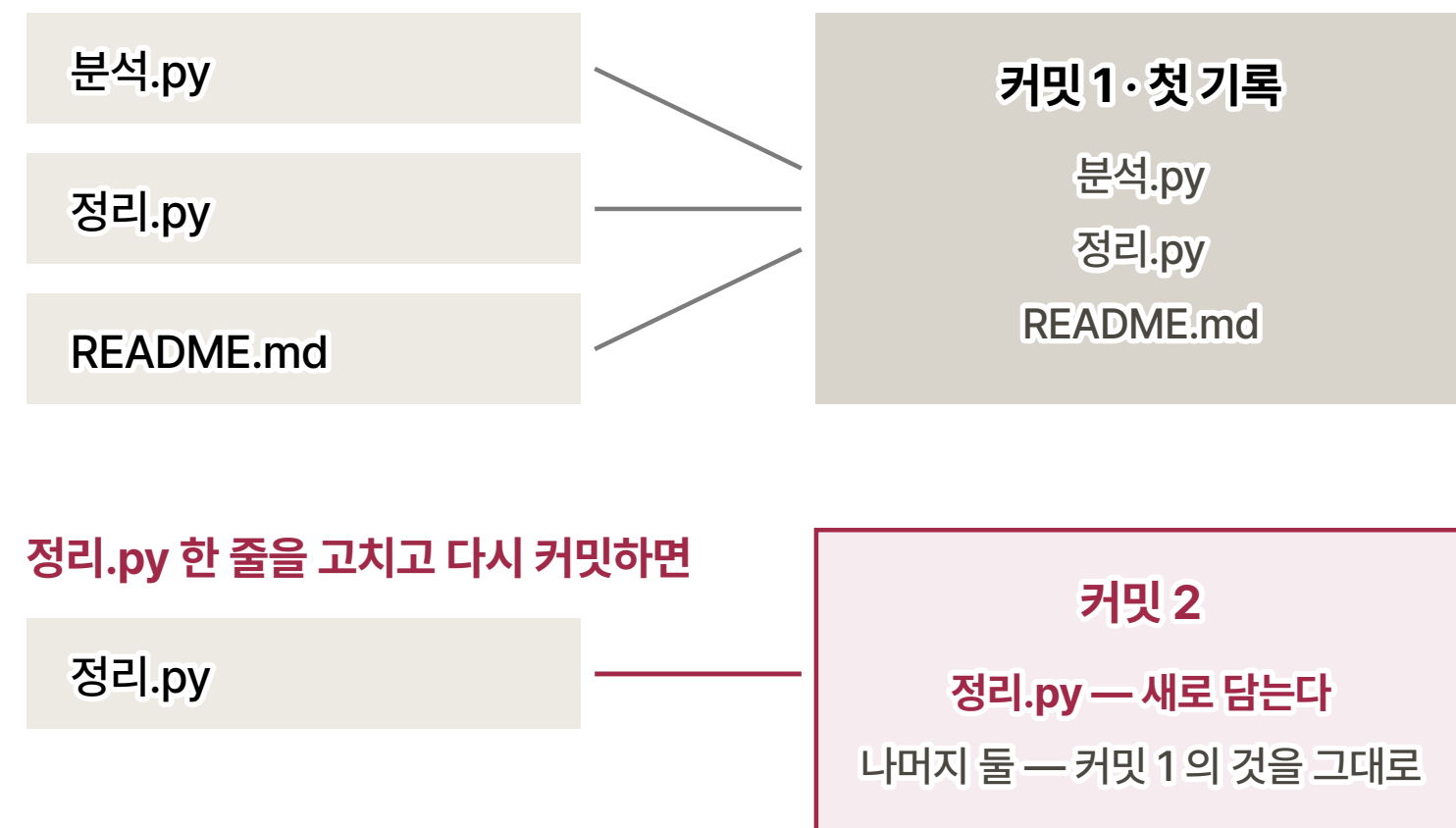


커밋은 "이 시점의 전부" 를 이름과 함께 남긴다

깃은 바뀐 것만 골라 쌓는다

작업 폴더

커밋



깃은 파일 이름이 아니라 **내용을 보고** 같은지 다른지 안다.

그래서 수천 개 파일 중 바뀐 것만 골라 담고,
나머지는 앞의 것을 다시 쓴다.

그래서 커밋이 쌓여도
저장소가 그만큼 무거워지지 않는다.

일단 이 넷이면 된다

터미널

`git init`이 폴더를 기록 대상으로

`git status`지금 뭐가 바뀌었나

`git add .`기록할 것을 고른다

`git commit`이름 붙여 남긴다

`-m "동별 요약 로직 확정"`

`git log --oneline`지금까지의 기록

`git diff`무엇이 바뀌었나

되돌리는 명령은 위험해서 여기 적지 않았다. **필요할 때 클로드에게 물어본다** — 무엇이 지워지는지 설명을 듣고 승인한다.

커밋 메시지를 이렇게 쓴다

이렇게 말고	이렇게
수정	면적 하한을 60m ² 로 정함
□ L O R	취소 거래 제외 규칙 추가
최종	동별 요약표 생성 로직

"무엇을 왜 바꿨는지" 를 적는다. 석 달 뒤에 이 목록만 읽고도 분석의 흐름이 보여야 한다.

넣지 않을 것 - .gitignore

넣지 않는다	왜
인증키 · 비밀번호	한 번 올라가면 지워도 기록에 남는다. 저장소를 공개로 바꾸는 순간 노출된다
큰 원본 데이터	수백 MB 를 커밋하면 저장소가 감당하지 못한다. <u>받는 방법을 적어 두면 된다</u>
개인정보가 든 자료	5주차에 말한 비공개 데이터가 여기 해당한다
다시 만들 수 있는 것	그림·PDF 처럼 스크립트로 재생성되는 산출물

.gitignore

```
.env          # 인증키
data/raw/     # 원본 데이터 - README 에 받는 법을 적는다
output/*.png  # 스크립트로 다시 만든다
```

첫 커밋 전에 .gitignore 부터 만든다. 올라간 뒤에 지우는 것은 훨씬 어렵다.

내 컴퓨터 밖에 두기

깃은 내 컴퓨터 안의 기록이다. 노트북이 고장 나면 함께 사라진다.

깃허브는 그 기록을 인터넷에 올려 두는 곳이다. 백업이자, 남에게 보여 주는 창구다.

터미널 · gh 를 설치하고 로그인한 뒤

gh auth login	한 번만
gh repo create	저장소 만들기
git push	올리기
git pull	내려받기

공개할 것과 하지 않을 것

이 수업이라면	
공개(public)	분석 스크립트 · 로직 문서 · 결과표 · README
비공개(private)	확정 전 초안, 개인정보가 섞였을 가능성이 있는 것

공개는 되돌리기 어렵다. 헛갈리면 비공개로 만들고 나중에 바꾼다 — 반대 방향은 이미 늦다.

COMPAS 비공개 데이터는 어느 쪽에도 올리지 않는다.

README – 재현의 설명서

남이 이 저장소를 열었을 때 **가장 먼저 보는 화면**이다.

이 수업의 기준은 하나다 — **이것만 읽고 내 결과를 다시 만들 수 있는가.**

원본 데이터를 올리지 않는 대신 **받는 방법을 적는다.** 그것이 5주차 "데이터 출처 문서"가 여기로 오는 자리다.

README.md

청주 상당구 실거래가 분석

무엇을 했나

동별 m²당 가격을 비교했다.

데이터

국토부 실거래가 API (공공데이터포털)

상당구 43111 · 2025.10~2026.09

받는 법: 인증키를 발급받아

`env`에 넣고 `python fetch.py`

분석 규칙

CLAUDE.md 참조 (60m² 초과 · 중앙값)

다시 만들려면

`python fetch.py` → `python summary.py`

결과는 `output/`에

기록하는 품을 맡긴다

클로드 코드는 터미널에서 돌므로 깃 명령을 그대로 쓸 수 있다

이렇게 시킨다

"지금까지 바뀐 걸 무엇을 왜 바꿨는지 알 수 있게 커밋해 줘. 커밋하기 전에 무엇을 담을지 목록으로 보여 줘."

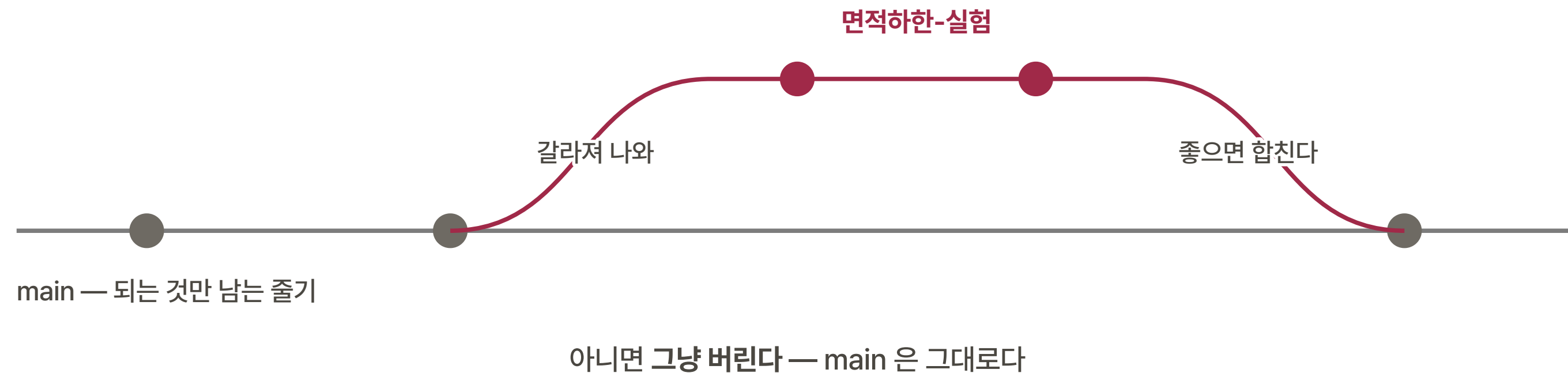
".env 는 빼고 커밋해."

"이 저장소의 커밋 기록을 읽고 분석이 어떻게 바뀌어 왔는지 정리해 줘."

굵게 표시한 부분이 핵심이다. **커밋 전에 목록을 보여 달라고 한다** — 무엇이 올라가는지 모르고 승인하면 인증키가 함께 간다.

마지막 줄은 이 수업다운 쓰임이다 — 커밋 기록 자체가 **분석 과정의 증거**가 된다.

브랜치 - 망칠 각오로 해 보기



깃에는 자리가 넷 있다

작업 폴더

분석.py

정리.py

README.md

커밋 대기

저장소 — 내 컴퓨터

깃허브 — 인터넷

깃에는 자리가 넷 있다

작업 폴더

분석.py

정리.py

README.md

커밋 대기

저장소 — 내 컴퓨터

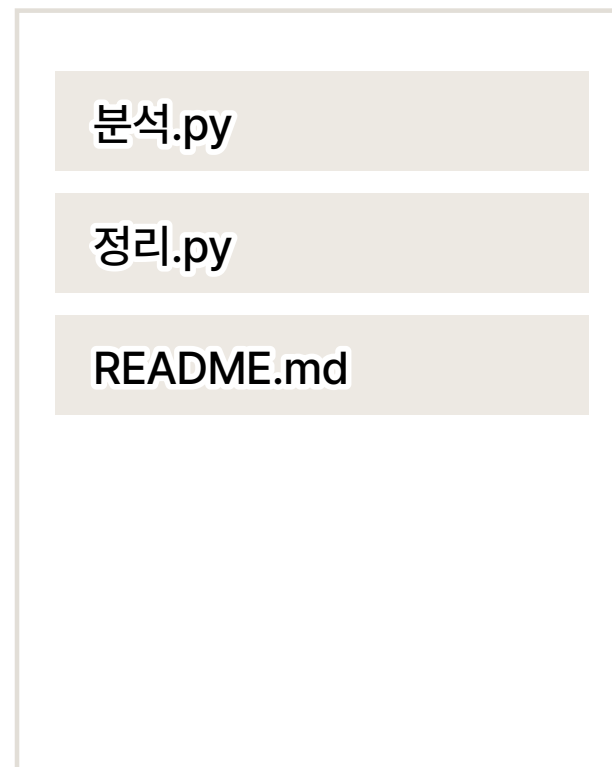
기록이 쌓이는 곳 — 폴더 안의 .git

깃허브 — 인터넷

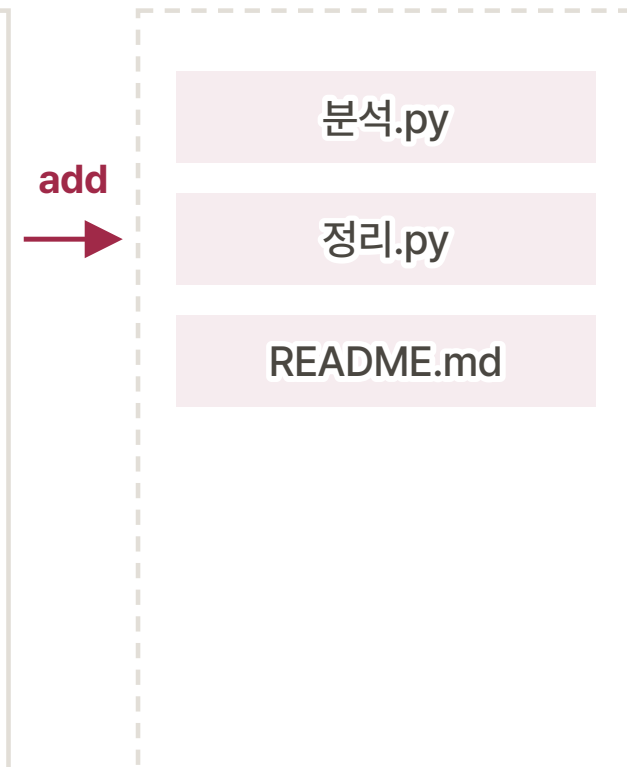
git init — 이 폴더를 기록 대상으로 삼는다

깃에는 자리가 넷 있다

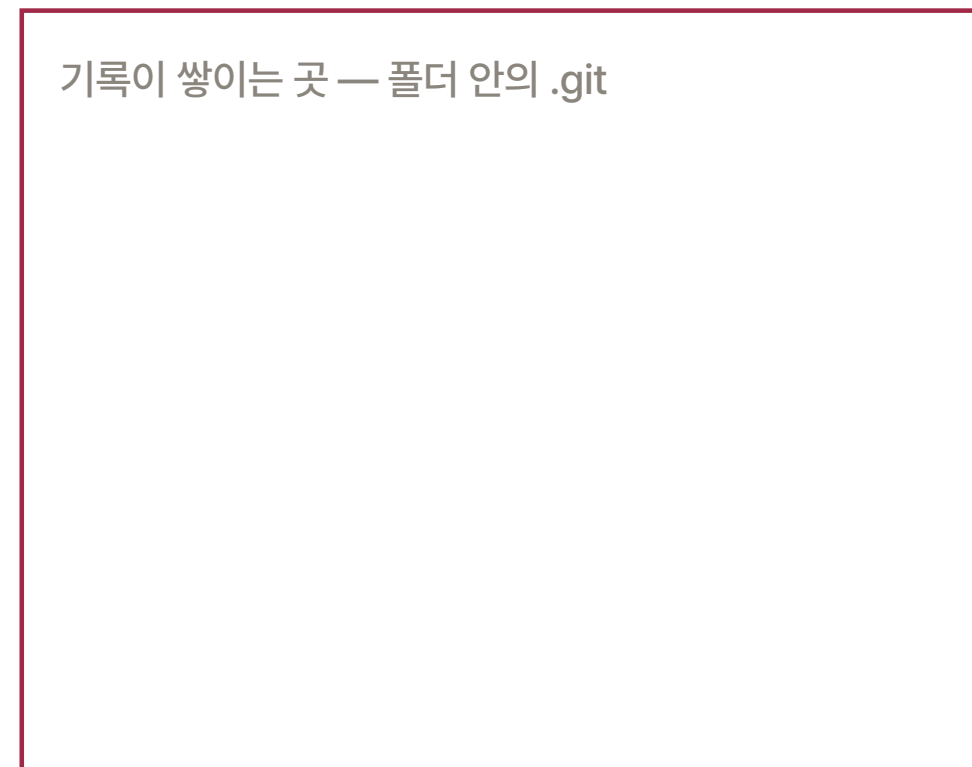
작업 폴더



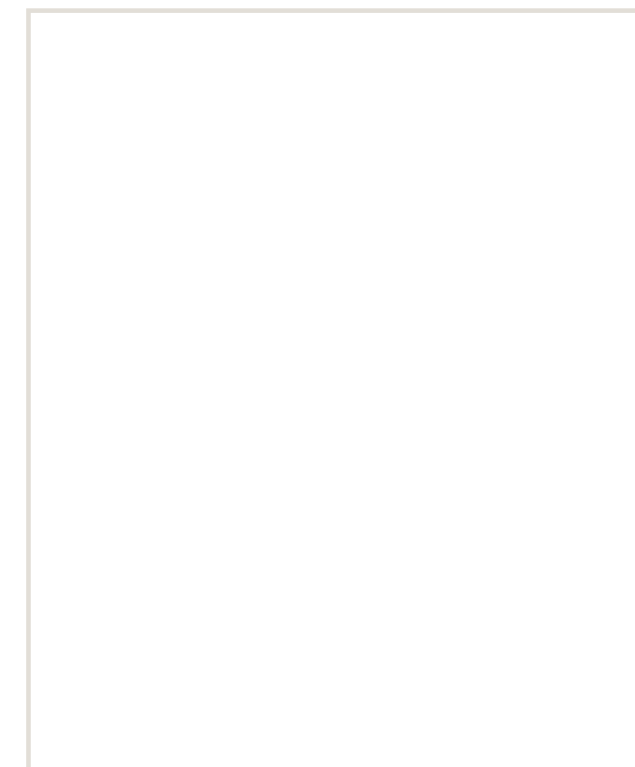
커밋 대기



저장소 — 내 컴퓨터



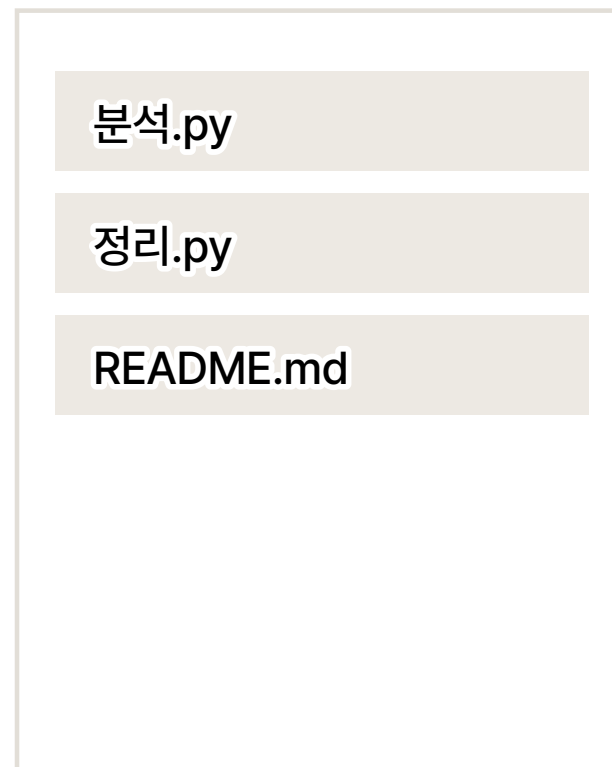
깃허브 — 인터넷



git add . — 이번 커밋에 담을 것을 골라 둔다. 파일은 그 자리에 그대로 있다

깃에는 자리가 넷 있다

작업 폴더



커밋 대기

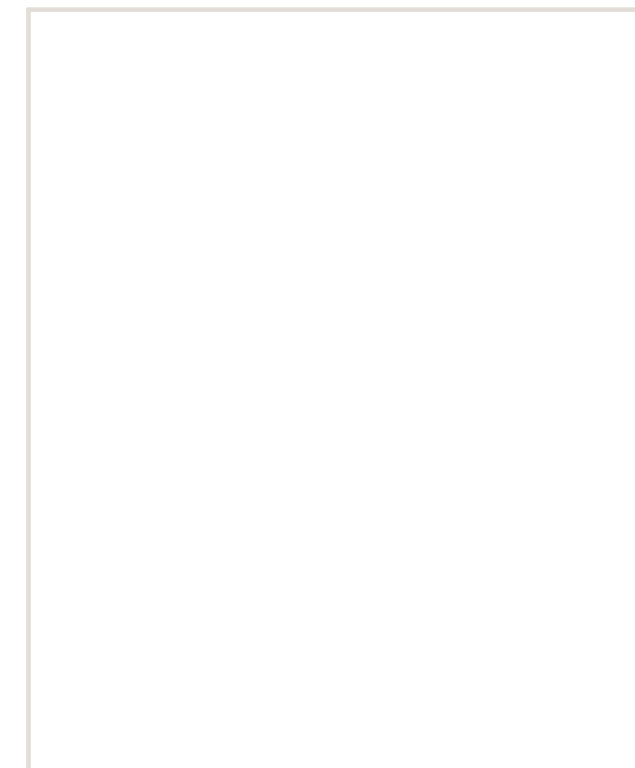


저장소 — 내 컴퓨터

commit

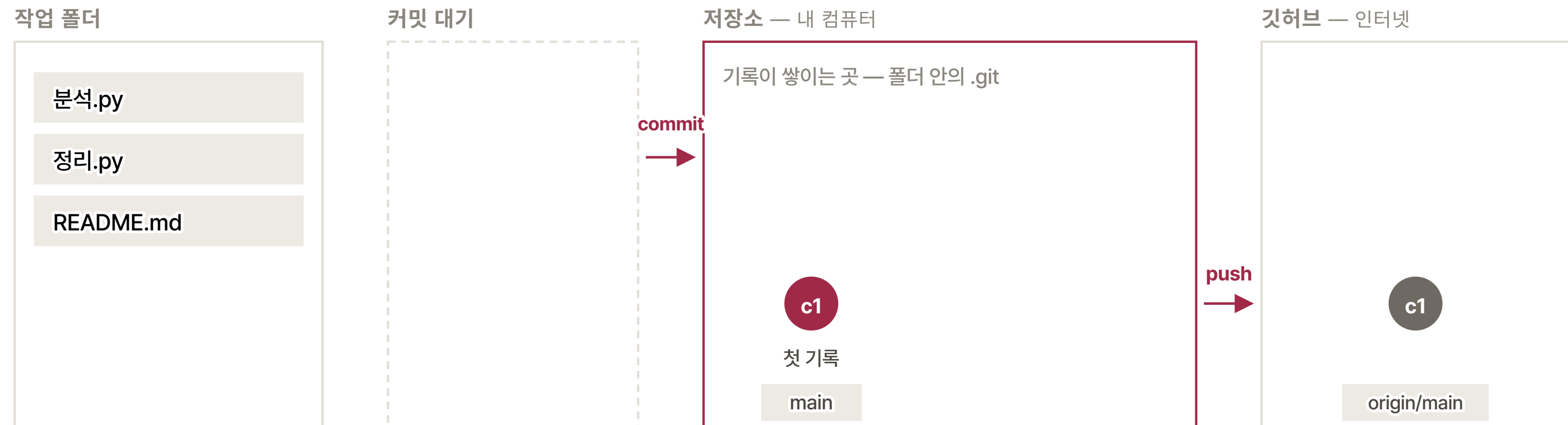


깃허브 — 인터넷



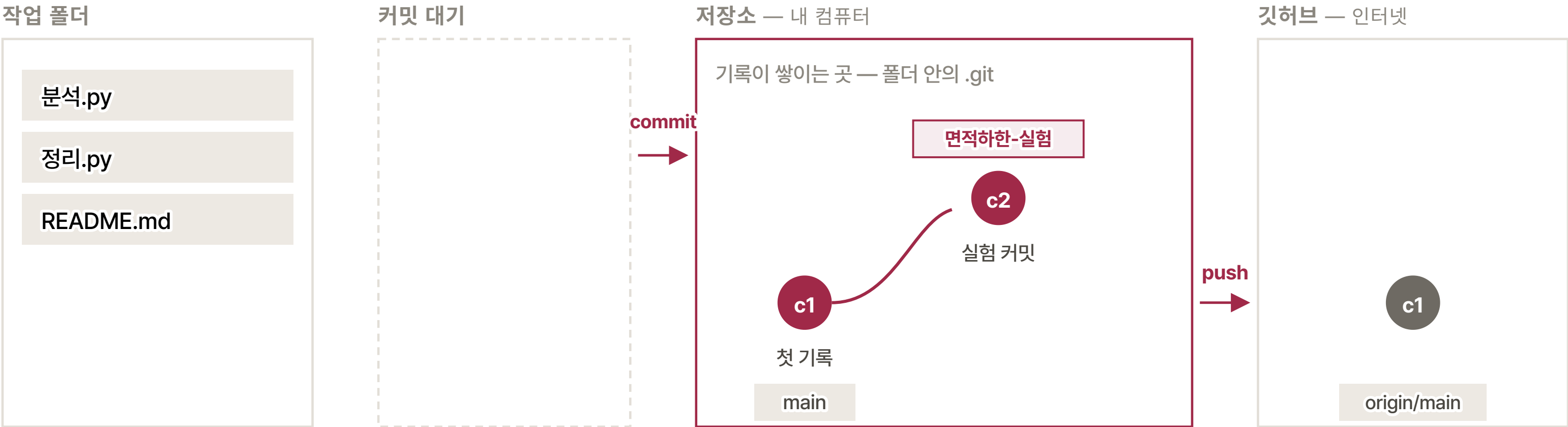
git commit -m "첫 기록" — 골라 둔 것을 이름 붙여 저장소에 남긴다

깃에는 자리가 넷 있다



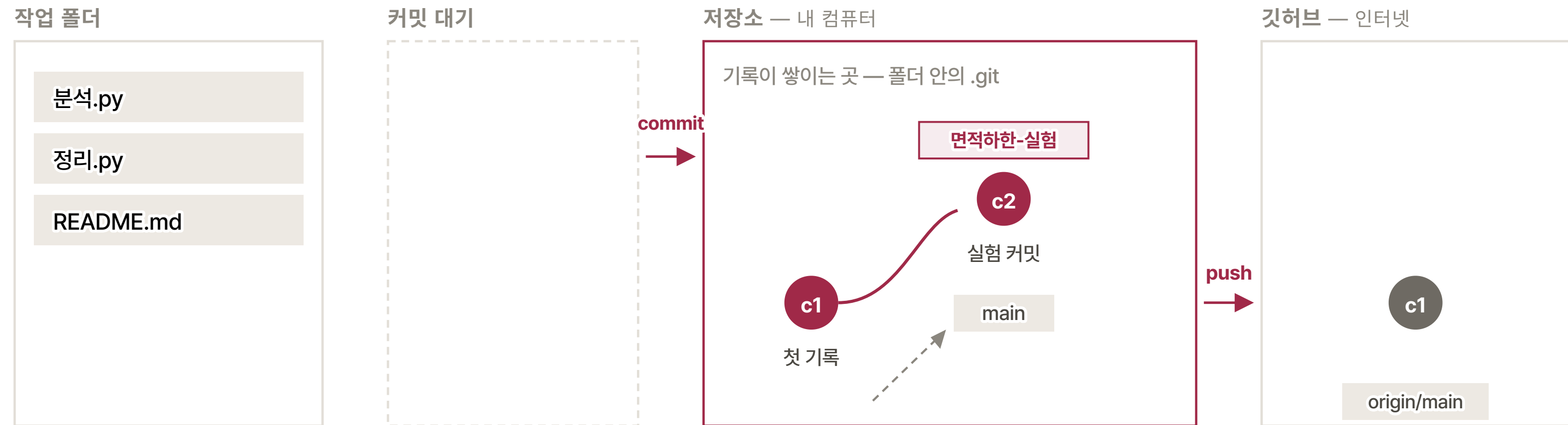
git push — 저장소의 커밋을 깃허브로 올린다

깃에는 자리가 넷 있다



`git switch -c 면적하한-실험` — main 을 두고 딴 줄기로 나가 커밋한다

깃에는 자리가 넷 있다



main 이 실험 자리까지 따라온다 — 새 커밋 없이 포인터만 옮겨 간다

git merge 면적하한-실험 — 실험이 쓸 만하면 main 으로 합친다

PR – 합치기 전에 보여 주기

풀 리퀘스트(PR)는 **"이렇게 바꾸려는데 봐 주세요"** 라는 요청이다. 바뀐 부분이 한눈에 보이고, 줄마다 의견을 달 수 있다.

터미널

```
git switch -c 면적하한-실험
# ... 작업 ...
git push -u origin 면적하한-실험
gh pr create
```

클로드에게 "브랜치 만들고 PR 까지 올려 줘" 라고 해도 된다. **올리기 전에 무엇을 담을지 확인한다.**

깃허브에서 클로드를 부르기

`/install-github-app` 을 실행해 두면 이슈나 PR 댓글에 **@claude** 로 부를 수 있다.

PR 댓글

@claude 이 분석 로직에서
빠뜨린 예외가 있는지 봐 줘

GitHub Actions 위에서 돌기 때문에 **저장소 관리자 권한**과 인증 설정이 필요하다. 팀으로 일할 때 쓰는 기능이라 오늘은 개념만 본다.

주의 – 이 수업에서 실제로 겪는 사고

사고	막는 법
인증키를 커밋했다	첫 커밋 전에 <code>.gitignore</code> 부터 만든다. 이미 올렸다면 그 키를 폐기하고 새로 발급받는다 — 기록에서 지우는 것보다 그게 빠르고 확실하다
비공개 데이터를 올렸다	COMPAS 비공개분·개인정보는 저장소에 넣지 않는다. <u>받는 방법만 적는다</u>
큰 파일로 저장소가 막혔다	원본 데이터는 <code>.gitignore</code> 로 뺀다
AI가 고친 걸 못 되돌린다	작업을 시작하기 전에 커밋해 둔다. 되돌릴 지점이 있어야 마음 놓고 시킨다

첫 줄이 가장 흔하고 가장 아프다. **공개 저장소는 검색된다.**

실습

내 분석을 저장소로 만들기 — 약 1시간 30분

오늘 실습의 도착점

남이 README 만 보고 내 결과를 다시 만들 수 있는 저장소

시각	할 일	끝났다는 신호
0:00-0:25	① 기록을 시작한다	첫 커밋이 남았다
0:25-0:55	② 브랜치로 실험하고 되돌린다	main 이 그대로 다
0:55-1:30	③ 서로 재현해 본다	남의 결과를 내가 만들었다

시작 전 확인

- 지난주 ds2026/ 폴더 (CLAUDE.md · 스킴 · 에이전트가 들어 있다)
- 깃 설치 · 깃허브 계정 · gh 설치

③은 짝이 필요하다. 홀수면 셋이 한 조로 돌려 가며 한다.

① 기록을 시작한다 0:00-0:25

- 1 `gh auth login` 으로 로그인한다 (브라우저가 열린다) 5분
- 2 `ds2026/` 에서 `git init` 1분
- 3 커밋하기 전에 `.gitignore` 를 만든다 8분
- 4 `git status` 로 올라갈 목록을 눈으로 읽는다 3분
- 5 `git add .` → `git commit -m "..."` 3분

3번을 이렇게 시킨다

"이 폴더를 깃 저장소로 만들려고 해. 먼저 어떤 파일이 올라가면 안 되는지 목록으로 알려 주고 `.gitignore` 초안을 보여 줘. 내가 확인하면 커밋하자."

✓ 확인 지점 — `git status` 목록에 인증키·원본 CSV 가 없다. 보이면 `.gitignore` 를 고치고 다시 본다.

② 실험하고 되돌린다 0:25-0:55

- 1 `git switch -c 면적하한-실험` 2분
- 2 CLAUDE.md 의 면적 하한을 **85m² 로 바꾸고** 분석을 다시 돌린다 10분
- 3 `git diff` 로 무엇이 바뀌었는지 보고 커밋한다 5분
- 4 `git switch main` 으로 돌아간다 2분
- 5 CLAUDE.md 를 열어 본다. 분석도 다시 돌려 본다 6분
- 6 실험을 살릴지 버릴지 정하고, 버릴 거면 브랜치를 지운다 5분

✓ 확인 지점 — 5번에서 CLAUDE.md 가 60m² 로 돌아와 있다. 결과표도 원래 값이다. 실험이 본 작업을 건드리지 않았다.

switch 가 거부되면 — 저장하지 않은 변경이 있는 것이다. 커밋하거나 되돌린 뒤 다시 한다. 무엇을 어떻게 할지는 클로드에게 물어보고 설명을 듣고 승인한다.

③-1 올리고 설명서를 쓴다 0:55-1:15

- 1 `gh repo create` 로 저장소를 만들고 올린다. **공개 여부를 스스로 정하고 이유를 한 줄 적는다** 7분
- 2 `README.md` 를 쓴다 10분
- 3 커밋하고 올린다. 브라우저로 저장소를 열어 눈으로 확인한다 3분

README 에 반드시 들어갈 넷

무엇을 했나	한두 문장
데이터 받는 법	출처 · 조건 · 인증키를 어디에 넣나 (5주차 출처 문서를 옮긴다)
분석 규칙	CLAUDE.md 를 가리키고 요약만
다시 만드는 순서	실행할 명령을 순서대로

✓ 확인 지점 — 브라우저에서 저장소를 열었을 때 인증키도 원본 데이터도 보이지 않는다.

③-2 서로 재현해 본다 1:15-1:30

- 1 옆 사람과 저장소 주소를 바꾼다 1분
- 2 상대 저장소를 받는다 — `git clone <주소>` 2분
- 3 **README 만 보고** 상대의 결과를 다시 만들어 본다. 막히면 물어보지 말고 그 지점을 적는다 8분
- 4 막힌 지점을 상대에게 알려 준다. 받은 지적으로 자기 README 를 고친다 4분

✓ 확인 지점 — 상대의 결과표와 **같은 숫자**가 나온다. 안 나오면 README 에 빠진 것이 있다는 뜻이고, **그 발견이** 오늘의 성과다.

시간이 모자라면 — 3번의 막힌 지점 목록까지만 하고 4번은 집에서. 일찍 끝났으면 상대 저장소에 브랜치를 만들어 README 를 고치고 PR 을 올려 본다.

제출물

무엇	어느 정도면 되나
저장소 주소	공개 여부와 그렇게 정한 이유 한 줄
커밋 기록	<code>git log --oneline</code> 결과. 메시지만 읽고 흐름이 보여야 한다
README	넷이 다 있고 ③-2 로 고친 흔적 이 있을 것
재현 후기	상대 저장소에서 어디서 막혔는지 와 내 README 를 어떻게 고쳤는지

마지막 줄이 이 강의의 핵심이다. **내가 아는 것과 적어 둔 것은 다르다** — 남이 막히는 자리에서만 그 차이가 드러난다.

다음 주부터 매주 실습 결과를 이 저장소에 커밋한다. 14주차에는 이 기록 전체가 증거 패키지의 바탕이 된다.

과제 – SGIS 스킬을 만들어 온다

3주차 과제는 내 일을 다뤘다. 이번엔 바깥 데이터다

SGIS(통계지리정보서비스)는 오픈API **93종**을 연다 — 인구·가구·주택·사업체 통계, 행정구역 경계, 지오코딩, 격자, 자연재해.

감쌀 것은 **두세 개**면 된다

- 1 **인증키를 오늘 신청한다** `sgis.mods.go.kr` · 사용기간을 학기 끝까지 덮게
- 2 자기 연구질문에 필요한 API 를 두세 개 고른다
- 3 클로드에게 스킬을 만들게 한다
- 4 실제로 받아 보고 **자릿수가 그럴듯한지** 본다

출처: 통계지리정보서비스 개발지원센터 `sgis.kostat.go.kr` (2026. 9. 6. 확인). API 93종 · 인증키당 하루 50,000회

보는 것 넷 — 분량은 세지 않는다

- 1 **스스로 꺼내지는가** description 이 「언제 쓰는지」를 말하는가
- 2 **인증키가 어느 파일에도 없는가** 이 폴더는 깃허브에 올라간다. 여기서 새면 키가 공개된다
- 3 **출처가 붙어 있는가** API 이름 · 기관 · 조회 날짜
- 4 **토큰 4시간을 사람이 안 만지는가**

마감은 6주차 수업 시작. 키 발급에 시간이 걸린다 — 사용기간을 짧게 잡으면 쓰기도 전에 만료된다.

자세한 것은 `과제-SGIS스킬.md` 에 있다. **6주차에 이 스킬로 자기 데이터를 받는다.**

자료 출처

- 깃 명령 — 공식 문서 `git-scm.com/doc`
- 깃허브 CLI — `cli.github.com`
- `/install-github-app` 과 `@claude` 멘션 — 클로드 코드 공식 문서 (2026. 8. 27. 확인)
`code.claude.com/docs/en/github-actions` · 저장소 관리자 권한과 인증 설정이 필요하다
- 5·11쪽 그림은 원리를 보이는 개념도다

Q & A

권규상 Kyusang Kwon
kyusang.kwon@chungbuk.ac.kr